

Lesson 12 ROS2 Parameter Instruction

1. Parameter Introduction

Similar to global variables in C++ programming, which can facilitate sharing certain data across multiple programs, parameters in the ROS robot system are global dictionaries that allow sharing data among multiple nodes.

In the ROS system, parameters are presented in the form of global dictionary. What is a dictionary? Just like a real dictionary, it consists of names and values, also known as keys and values, forming key-value pairs.

Alternatively, it can also be understood as the parameters in programming, where there is a parameter name followed by an equal sign, and then the parameter value. When using it, you can access this parameter name.

Parameters have rich features. For example, if a node shares a parameter, other nodes can access it. If a node modifies a parameter, other nodes can also immediately know about it, thus obtaining the latest value.

2 .param Parameter

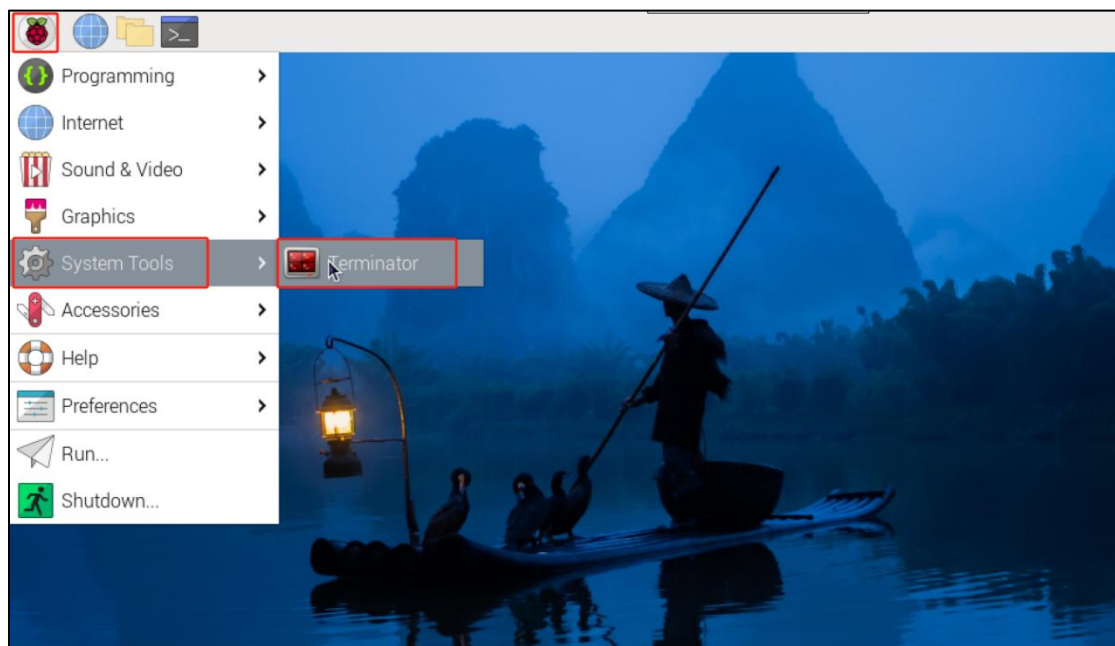
Let's first get to know param parameter commands. The detailed instruction is as follows:

Command	Instruction
ros2 param list	List current multiple parameters
ros2 get param_key	List a certain parameter value

ros2 set param_key param_value	Set a certain parameter value
ros2 param dump file_name	Save parameters into file
ros2 param load file_name	Read parameters from a file
ros2 param delete param_key	Delete parameters

3. Create A Parameter Example

- 1) Click  and select "System Tools -> Terminator" in sequence.



- 2) Enter the command "Enter the command **cd hiwonder_ws/src/demo_interfaces**" to switch to the src folder within the hiwonder_was workspace.

```
ubuntu@raspberrypi:~$ cd hiwonder_ws/src/  
ubuntu@raspberrypi:~/hiwonder_ws/src$
```

3) Enter the command “**ros2 pkg create param_demo --build-type ament_python --dependencies rclpy**” and press Enter to create a package named “param_demo” with rclpy dependency.

```
ubuntu@raspberrypi:~/hiwonder_ws/src$ ros2 pkg create param_demo --build-type ament_python --dependencies rclpy
```

4) Enter the command “**cd param_demo/param_demo/**” to switch to the “param_demo” package.

```
ubuntu@raspberrypi:~/hiwonder_ws/src$ cd param_demo/param_demo/  
ubuntu@raspberrypi:~/hiwonder_ws/src/param_demo/param_demo$
```

5) Enter the command “**vim param_demo.py**” to open the file using VIM text editor. Copy the program below. If you need to make modifications, you can press “i” to enter the insert mode. If you’ve finishing the modification, press “Esc” and enter “:wq” to save and exit.

```
ubuntu@raspberrypi:~/hiwonder_ws/src/param_demo/param_demo$ vim param_demo.py
```

```
import rclpy # Import rclpy module  
  
from rclpy.node import Node # Import Node class  
  
from rclpy.parameter import Parameter # Import Parameter class  
  
class MinimalParam(Node): # Define the MinimalParam class inhering  
from the Node class  
  
    def __init__(self):  
  
        super().__init__('minimal_param_node') # Call the constructor  
of the parent class to initialize the node.
```

```
self.declare_parameter('my_parameter', 'hiwonder') # Declare
a parameter named "my_parameter" and set its default value to "hiwonder".

self.timer = self.create_timer(1, self.timer_callback) # Create a
time, set the callback function as timer_callback, with an interval of 1 second.

def timer_callback(self):

    my_param =
self.get_parameter('my_parameter').get_parameter_value().string_value
#Retrieve the value of parameter 'my_parameter', and convert it to a string.

    self.get_logger().info('Hello %s!' % my_param) # Print the log
message with parameter values

    my_new_param = Parameter( # Create a new parameter
object
        'my_parameter', # The parameter name is 'my_parameter'.
        rclpy.Parameter.Type.STRING, # The parameter type is
string
        'hiwonder' # The parameter value is 'hiwonder'
    )

    all_new_parameters = [my_new_param] # Place the new
parameter object into the list.

    self.set_parameters(all_new_parameters) # Set node's
parameter value as new parameter value.
```

```
def main():

    rclpy.init() # Initialize ROS node

    node = MinimalParam() # Create MinimalParam object

    rclpy.spin(node) # Enter main loop


if __name__ == '__main__':

    main()
```

```
if __name__ == '__main__':
    # 执行主函数
    main()
:wq
```

6) Enter the command “**chmod +x param_demo.py**” and press Enter to grant the executable permission to the saved **param_demo.py** file.

```
ubuntu@raspberrypi:~/hiwonder_ws/src/param_demo/param_demo$ chmod +x param_demo.py
ubuntu@raspberrypi:~/hiwonder_ws/src/param_demo/param_demo$
```

4. Compilation and Execution

1) After granting the executing permission, enter the command “**cd ~/hiwonder_ws/**” to switch to the directory of the workspace.

```
ubuntu@raspberrypi:~/hiwonder_ws/src/param_demo/param_demo$ cd ~/hiwonder_ws/
ubuntu@raspberrypi:~/hiwonder_ws$
```

2) Enter the command “**colcon build**” and press Enter to compile the packages within the workspace.

```
ubuntu@raspberrypi:~/hiwonder_ws$ colcon build
```

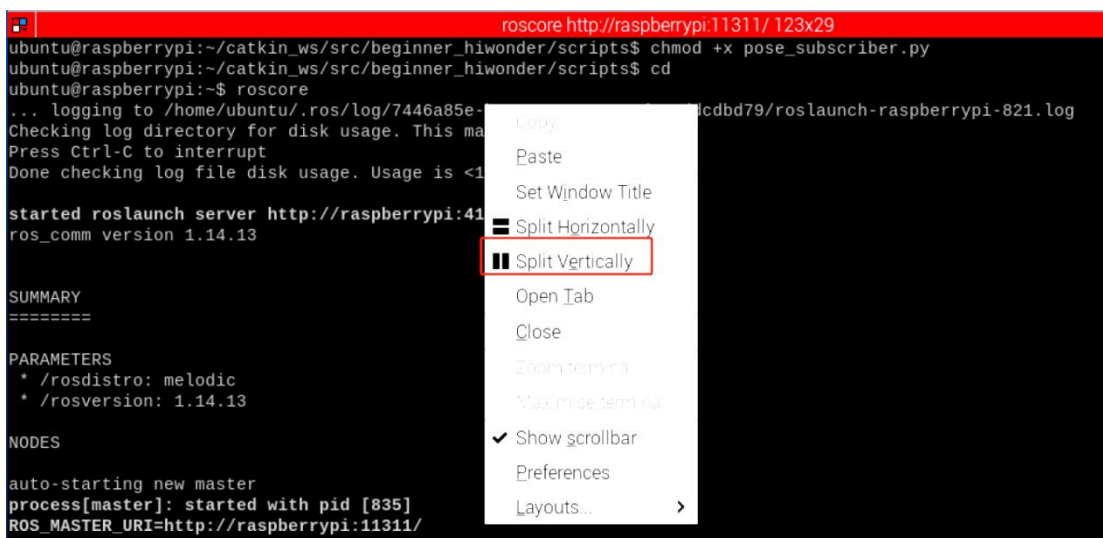
3) Enter the command “**source ./install/setup.bash**” and press Enter to make the environment variables take effect.

```
ubuntu@raspberrypi:~/hiwonder_ws$ source ./install/setup.bash
```

4) Enter the command “**ros2 run param_demo param_demo**” and press Enter to start the param_demo node.

```
ubuntu@raspberrypi:~$ ros2 run param_demo param_demo
[INFO] [1706322737.926459775] [minimal_param_node]: Hello hiwonder!
[INFO] [1706322738.914357462] [minimal_param_node]: Hello hiwonder!
[INFO] [1706322739.914370172] [minimal_param_node]: Hello hiwonder!
[INFO] [1706322740.914414199] [minimal_param_node]: Hello hiwonder!
```

5) Right click to select “**Split Vertically**” to create a new terminal window.



```
roscore http://raspberrypi:11311/ 123x29
ubuntu@raspberrypi:~/catkin_ws/src/beginner_hiwonder/scripts$ chmod +x pose_subscriber.py
ubuntu@raspberrypi:~/catkin_ws/src/beginner_hiwonder/scripts$ cd
ubuntu@raspberrypi:~$ roscore
... logging to /home/ubuntu/.ros/log/7446a85e-...
Checking log directory for disk usage. This ma...
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1...
started roslaunch server http://raspberrypi:41...
ros_comm version 1.14.13

SUMMARY
=====

PARAMETERS
* /rostdistro: melodic
* /rosversion: 1.14.13

NODES
auto-starting new master
process[master]: started with pid [835]
ROS_MASTER_URI=http://raspberrypi:11311/

[Context Menu]
- Copy
- Paste
- Set Window Title
- Split Horizontally
- Split Vertically
- Open Tab
- Close
- Zoom terminal
- Maximize terminal
- Show scrollbar (checked)
- Preferences
- Layouts... >
```

6) Enter the command “**ros2 param set minimal_param_node my_parameter world**” and press Enter to modify the parameter my_parameter of the node minimal_param_node to “world”.

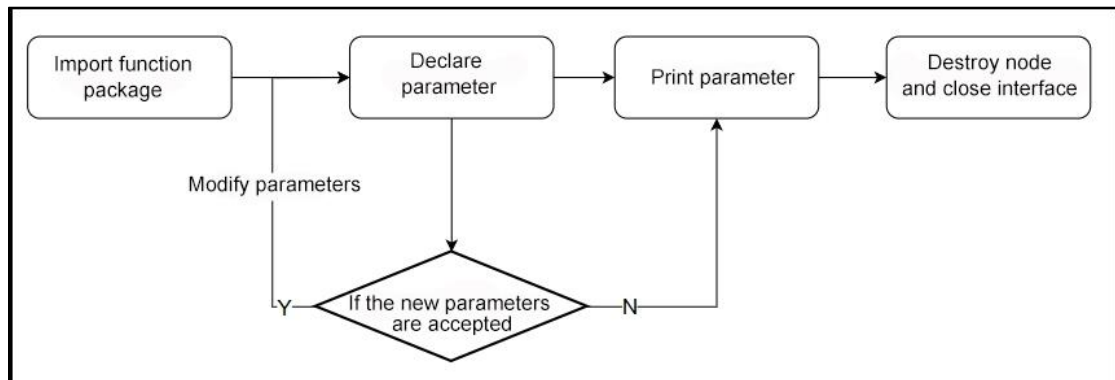
```
ubuntu@raspberrypi:~$ ros2 param set minimal_param_node my_parameter world
Set parameter successful
```

At this point, you can notice that the output from the param_demo node is “**Hello world!**”, indicating the parameter are modified successfully.

```
[INFO] [1706322934.672806231] [minimal_param_node]: Hello hiwonder!
[INFO] [1706322935.672741228] [minimal_param_node]: Hello hiwonder!
[INFO] [1706322936.672833874] [minimal_param_node]: Hello hiwonder!
[INFO] [1706322937.673462392] [minimal_param_node]: Hello world!
[INFO] [1706322938.672808951] [minimal_param_node]: Hello hiwonder!
[INFO] [1706322939.672779104] [minimal_param_node]: Hello hiwonder!
[INFO] [1706322940.672838369] [minimal_param_node]: Hello hiwonder!
```

5. Program Analysis

According to the realization result, the logic progress for the program is shown as pictured:



Create a MinimalParam class, declare a string parameter "my_parameter" in the constructor, and create a timer with a 1-second interval. The timer callback function is called every 1 second. Inside the callback, it first retrieves the current value of the parameter, then prints a log, creates a new parameter object, sets a new value, and finally calls the set_parameters function to modify the parameter value in the parameter server.

◆ Main Function

```
def main():
    rclpy.init() # 初始化ROS节点
    node = MinimalParam() # 创建MinimalParam对象
    rclpy.spin(node) # 进入主循环
```

First, invoke the `rclpy.init()` function to initialize ROS2 Python interface. Then instantiate the **MinimalParam**. Finally, execute the `node.run()` function.

◆ MinimalParam Class

```
class MinimalParam(Node): # 定义一个继承自Node类的MinimalParam类
    def __init__(self):
        super().__init__('minimal_param_node') # 调用父类构造函数初始化节点

        self.declare_parameter('my_parameter', 'hiwonder') # 声明一个名为'my_parameter'的参数,并设置默认值为'hiwonder'

        self.timer = self.create_timer(1, self.timer_callback) # 创建定时器,设置回调函数为timer_callback,时间间隔为1秒

    def timer_callback(self):
        my_param = self.get_parameter('my_parameter').get_parameter_value().string_value # 获取参数'my_parameter'的值,并转换为字符串

        self.get_logger().info('Hello %s!' % my_param) # 打印带有参数值的日志消息

        my_new_param = Parameter( # 创建一个新的参数对象
            'my_parameter', # 参数名称为'my_parameter'
            rclpy.Parameter.Type.STRING, # 参数类型为字符串
            'hiwonder' # 参数值为'hiwonder'
        )
        all_new_parameters = [my_new_param] # 将新的参数对象放入列表中
        self.set_parameters(all_new_parameters) # 设置节点的参数值为新的参数值
```

First, a string parameter named "**my_parameter**" is declared, and a timer with a 1-second period is created. The timer callback function, `timer_callback`, is called every second. It first retrieves the current value of the "**my_parameter**" parameter and then prints a log output. Next, the callback function creates a new parameter object with the same name but with a value of "hiwonder". Afterward, it calls the `set_parameters` function to modify the parameter value.